

Universidad Nacional de Córdoba

FACULTAD DE MATEMÁTICA, ASTRONOMÍA, FÍSICA Y COMPUTACIÓN



Sistema de Control PID en Tiempo Real para Balanceo
Informe final de proyecto

SISTEMAS DE TIEMPO REAL

Sferco, Martín

11 de septiembre de 2026

Índice

1	Resumen del proyecto	2
2	Objetivos del Proyecto	2
2.1	Objetivo General	2
2.2	Objetivos Específicos	2
3	Descripción funcional del sistema	3
3.1	Diagrama de interacción	3
3.2	Entradas y Salidas del Sistema	3
3.2.1	Entradas	3
3.2.2	Salida	3
4	Arquitectura de Software y Diseño en FreeRTOS	3
4.1	Identificación de Tareas	4
4.2	Asignación de Prioridades	5
4.3	Sincronización y Comunicación entre Tareas (IPC)	5
4.4	Manejo de Interrupciones (ISRs)	5
4.5	Cambios realizados en la sección	6
5	Organización del proyecto	6
5.1	Carpetas generadas por CubeMX	6
5.2	Carpetas propias de la aplicación	7
6	Grado de avance	7
6.1	¿Se finalizó el desarrollo completo?	7
6.2	Funcionamiento correcto de los periféricos	7
6.3	Modelado sobre RTOS	8
7	Evolución del proyecto	8
8	Métricas	9
9	Construcción de la Maqueta	10
10	Conclusiones	11

1. Resumen del proyecto

El sistema consiste en una barra horizontal articulada en uno de sus extremos, sobre la cual se desliza libremente un objeto deslizante. Mediante un motor, el extremo libre de la barra puede elevarse o descender, generando una inclinación que provoca el desplazamiento del objeto a lo largo de ella.

El objetivo del sistema es posicionar el objeto deslizante en un punto específico de la barra y mantenerlo estable en esa posición, controlando la inclinación a través de un algoritmo de control PID. La posición deseada puede ser ajustada en cualquier momento por el usuario a través de un periférico de entrada.

Para medir la posición actual del objeto sobre la barra se utilizará un sensor de distancia. El error entre la posición medida y la posición deseada es procesado por el algoritmo PID, que genera una señal de corrección para actuar sobre el servo motor, modificando la inclinación de la barra para reducir dicho error. Para definir la posición deseada, el usuario cuenta con un potenciómetro que permite ajustar el setpoint del sistema. El sistema de control será implementado sobre un microcontrolador STM32-F401RE, utilizando el sistema operativo de tiempo real FreeRTOS para gestionar la concurrencia y garantizar el cumplimiento de los requisitos temporales del sistema.

Todo el código del proyecto se encuentra disponible en el siguiente **repositorio de GitHub**.

2. Objetivos del Proyecto

2.1. Objetivo General

El propósito general de este trabajo es el de implementar un sistema de control PID en tiempo real sobre un microcontrolador STM32-F401RE con FreeRTOS, capaz de posicionar y mantener un objeto deslizante en una posición determinada sobre una barra articulada, actuando sobre la inclinación de la misma de manera precisa, eficiente, y en el menor tiempo posible.

2.2. Objetivos Específicos

Para poder alcanzar dicho objetivo general deberemos conseguir determinados objetivos específicos que iremos enumerando a continuación. Nosotros queremos:

- **Muestreo periódico del sensor ultrasónico:** Configurar y leer el sensor HC-SR04 con un período fijo de 100 ms, obteniendo la posición actual del objeto deslizante de forma periódica y confiable.
- **Filtrado de señal con filtro de Kalman:** Implementar un filtro de Kalman sobre las lecturas del HC-SR04 para reducir el ruido del sensor ultrasónico y mejorar la precisión en la estimación de posición.
- **Implementación del algoritmo PID:** Desarrollar el algoritmo de control PID que calcule la señal de corrección a partir del error entre la posición medida y la posición deseada.
- **Generación de señal PWM para el servo motor:** Generar una señal PWM mediante un timer del STM32 para controlar la posición angular del servo motor y modificar así la inclinación de la barra.
- **Lectura periódica del potenciómetro y conversión ADC:** Leer periódicamente el potenciómetro lineal a través del conversor ADC del microcontrolador y mapear el valor obtenido a una posición objetivo expresada en centímetros sobre la barra.

3. Descripción funcional del sistema

3.1. Diagrama de interacción

En la Figura 1 se presenta el diagrama general del sistema, mostrando cómo interactúan los distintos componentes entre sí y con el mundo físico.

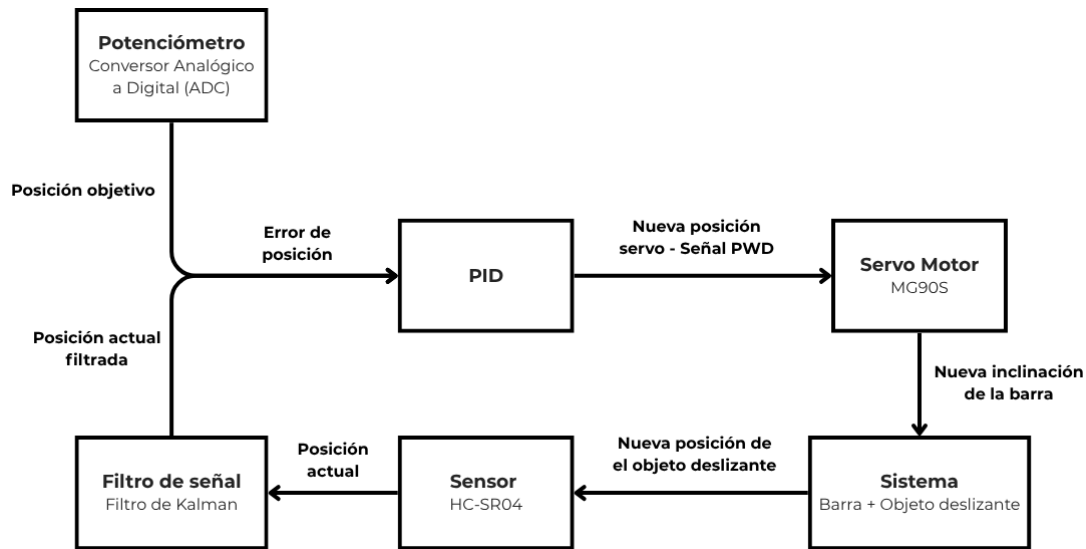


Figura 1: Diagrama de interacción del sistema con el mundo físico.

3.2. Entradas y Salidas del Sistema

3.2.1. Entradas

- **Potenciómetro lineal:** Señal analógica que representa la posición deseada del objeto sobre la barra. Obtenemos una medición normalizada desde el ADC del microcontrolador y luego la mapeamos a una posición objetivo expresada en centímetros.
- **Sensor HC-SR04:** Permite determinar la posición actual del objeto deslizando sobre la barra, entregando una medición de distancia que el sistema utiliza como variable de realimentación.

3.2.2. Salida

- **Señal PWM:** Señal de control hacia el servo motor que determina su ángulo y, por ende, la inclinación de la barra.

4. Arquitectura de Software y Diseño en FreeRTOS

En la Figura 2 se puede observar el diagrama completo de tareas y su interacción con el hardware y entre sí.

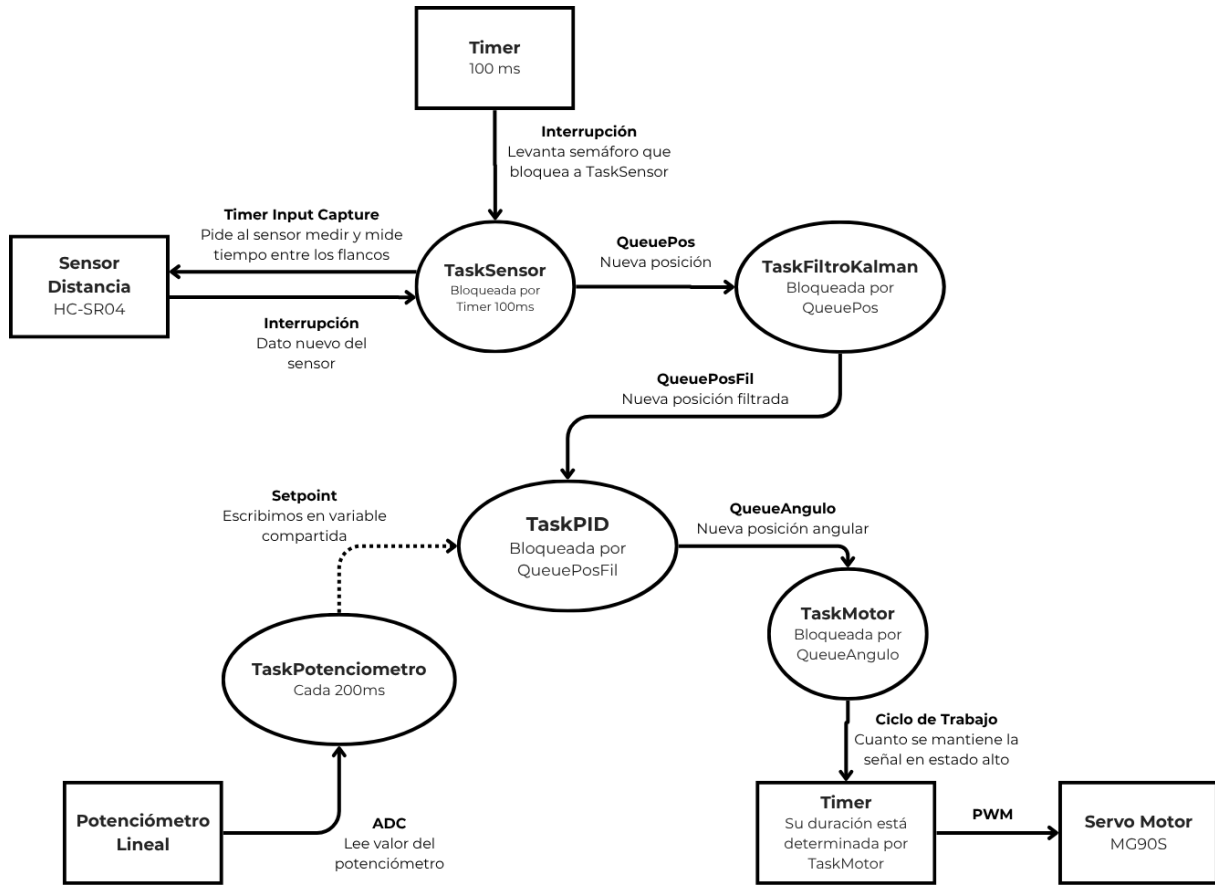


Figura 2: Diagrama de tareas

4.1. Identificación de Tareas

El sistema se estructura en cinco tareas concurrentes, cada una con una responsabilidad acotada. Para mayor detalle, analizar la Figura 2.

- **TaskSensor:** Responsable del ciclo de medición del sensor HC-SR04. Se despierta cada 100 ms mediante un semáforo liberado por el timer de hardware, dispara el pulso de trigger y espera la notificación de la interrupción del TIM_IC con la distancia medida.
- **TaskFiltroKalman:** Recibe la posición cruda desde TaskSensor y aplica el filtro de Kalman para reducir el ruido de las mediciones del sensor ultrasónico, entregando una posición filtrada a TaskPID.
- **TaskPID:** Ejecuta el algoritmo PID. Se bloquea esperando cualquier evento proveniente de QueuePosFil, recalculando la señal de corrección ante cualquier cambio en la posición filtrada.
- **TaskMotor:** Recibe la señal de corrección calculada por TaskPID y actualiza el registro de ciclo de trabajo del timer PWM de hardware, que genera la señal hacia el servo motor de forma autónoma.
- **TaskPotenciometro:** Lee periódicamente el valor del potenciometro lineal mediante el ADC y actualiza el setpoint del sistema. Se ejecuta cada 200 ms utilizando `vTaskDelayUntil`. También se podría usar otro timer como el que usamos para TaskSensor.

4.2. Asignación de Prioridades

La asignación de prioridades se realizó considerando el impacto de cada tarea sobre la estabilidad del sistema de control. Como se observa en la Figura 2, TaskSensor es el punto de entrada de toda la cadena, por lo que un retraso en ella se propaga inevitablemente al resto.

Tarea	Prior.	Justificación
TaskSensor	5	Punto de entrada de la cadena. Un retraso se propaga a todas las tareas siguientes.
TaskFiltroKalman	4	Debe ejecutarse tras cada medición. Sin su resultado, TaskPID no puede operar.
TaskMotor	4	Actualización oportuna del actuador. Opera en rama independiente a TaskFiltroKalman y es de muy corta duración.
TaskPID	3	Depende de TaskFiltroKalman, por lo que tiene menor prioridad que esta.
TaskPotenciometro	1	Interacción del usuario (firm real-time). Es la de menor prioridad del sistema: puede esperar sin afectar el control.

Cuadro 1: Prioridades de las tareas y su justificación.

4.3. Sincronización y Comunicación entre Tareas (IPC)

Los mecanismos de comunicación entre tareas se pueden observar en la Figura 2. Se detallan a continuación:

- **Queues de tamaño 1:** Se utilizarán tres queues, todas de profundidad 1, garantizando que cada tarea trabaje siempre con el dato más reciente:
 - QueuePos: transfiere la posición cruda desde TaskSensor hacia TaskFiltroKalman.
 - QueuePosFil: transfiere la posición filtrada desde TaskFiltroKalman hacia TaskPID.
 - QueueAngulo: transfiere la señal de corrección desde TaskPID hacia TaskMotor.
- **Setpoint sin queue:** TaskPotenciometro no dispara ningún recálculo del algoritmo PID por si solo, así que el setpoint se transmite como una variable compartida a la que TaskPID accede por un puntero.
- **Semáforos binarios:** Se utilizarán dos semáforos binarios en la cadena de medición:
 - SemTimer: liberado por la ISR del timer de 100 ms para despertar TaskSensor al inicio de cada ciclo de medición.
 - SemSensor: liberado por la ISR del TIM_IC para notificar a TaskSensor que la medición está lista y el dato puede ser leído.

4.4. Manejo de Interrupciones (ISRs)

El sistema requiere dos interrupciones de hardware, ambas visibles en la Figura 2:

- **TIM_Update:** Genera una interrupción cada 100 ms que libera un semáforo que desbloquea a TaskSensor para iniciar un nuevo ciclo de medición.

- **TIM_IC (Input Capture):** Captura los flancos del pin de Echo del HC-SR04 por hardware. Al completar la medición, libera un semáforo notificando a TaskSensor que el dato está disponible.

4.5. Cambios realizados en la sección

A diferencia de lo reportado en una etapa temprana del proyecto, la arquitectura de tiempo real descrita a lo largo de esta sección ya se encuentra completamente implementada: las cinco tareas, las tres colas, el setpoint como variable compartida entre TaskPotenciómetro y TaskPID, los dos semáforos binarios y las dos interrupciones de hardware están integrados y funcionando en conjunto sobre el microcontrolador, formando un lazo de control cerrado real.

Respecto del diseño original planteado en el anteproyecto, se introdujeron los siguientes ajustes durante la implementación:

- Todas las colas, semáforos y tareas se crearon utilizando las variantes **Static** de FreeRTOS (memoria reservada en tiempo de compilación); el sistema no utiliza heap en ningún punto.
- La prioridad de TaskPotenciómetro se fijó en 1 en lugar de 0, ya que la prioridad 0 está reservada en FreeRTOS (*idle task*).
- El diseño original bloqueaba a TaskPID en un Queue Set compuesto por QueuePosFil y QueueObjetivo. Se reemplazó por un único bloqueo en QueuePosFil, publicando el setpoint del potenciómetro en una variable compartida en lugar de una cola, tal como se detalla en la subsección anterior y se justifica en la sección de evolución del proyecto.

Sobre el sensor ultrasónico HC-SR04 y el servomotor MG90S en particular, ambos periféricos ya fueron validados individualmente y luego integrados a la cadena completa. Durante esa integración surgieron ajustes adicionales, no contemplados originalmente, que se documentan con mayor detalle en la sección de evolución del proyecto: la definición de una única referencia geométrica de borde para todo el lazo de control, y una política de recorte de las lecturas del sensor que distingue entre un eco real fuera de rango y un eco inválido por zona ciega o timeout.

5. Organización del proyecto

El proyecto se desarrolla sobre STM32CubeIDE, por lo que buena parte de su estructura de carpetas es generada automáticamente por la herramienta CubeMX a partir de la configuración de periféricos. El código propio de la aplicación se organizó en carpetas separadas de las generadas, de modo que el proyecto pueda regenerarse desde CubeMX sin perder ni pisar el trabajo propio.

5.1. Carpetas generadas por CubeMX

- **Core/:** código de arranque e inicialización de periféricos (**main.c**, configuración de timers, ADC, GPIO, manejadores de interrupción, etc.). Salvo un mínimo de código agregado a mano en **main.c** y en la configuración de FreeRTOS, esta carpeta es enteramente generada por CubeMX y se regenera automáticamente al modificar la configuración gráfica de periféricos.
- **Drivers/:** capa HAL de STM32 y CMSIS, provista por el fabricante. No contiene código propio.
- **Middlewares/:** el kernel de FreeRTOS, provisto como biblioteca de terceros. Tampoco contiene código propio.

5.2. Carpetas propias de la aplicación

El código propio se organiza en cuatro carpetas, cada una con una responsabilidad bien delimitada dentro del sistema:

- **App/**: punto de composición de la aplicación. Contiene `app_config.h`, único lugar donde se definen las constantes del sistema (parámetros físicos del montaje, ganancias del PID y del filtro de Kalman, prioridades y tamaños de pila de las tareas), y `app.c`, donde se instancian los drivers, se crean las colas y los semáforos, y se lanzan las cinco tareas de FreeRTOS descritas en la Figura 2.
- **ComponentDrivers/**: drivers de los periféricos de hardware que dependen de la capa HAL del microcontrolador: el sensor ultrasónico HC-SR04, el servomotor MG90S y el potenciómetro lineal. Cada uno se implementa de forma independiente de FreeRTOS, exponiendo una interfaz simple (por ejemplo, ángulo en grados para el servo, o valor normalizado para el potenciómetro).
- **Control/**: algoritmos de control puros, sin ninguna dependencia de la capa HAL ni de FreeRTOS: el filtro de Kalman, el controlador PID y una utilidad de mapeo lineal para convertir la lectura del potenciómetro a centímetros. Al no depender del hardware, se pueden probar de forma aislada.
- **Tasks/**: una tarea de FreeRTOS por cada etapa del lazo de control (sensor, filtro de Kalman, PID, motor y potenciómetro), donde cada tarea conecta los drivers de **ComponentDrivers/** y los algoritmos de **Control/** a través de los mecanismos de sincronización definidos en **App/**.

Esta separación entre lo generado por CubeMX, los drivers de hardware, los algoritmos de control y las tareas que los conectan permite que el sistema sobreviva a una regeneración del proyecto desde CubeMX, y facilita distinguir qué partes del código son específicas del microcontrolador utilizado y cuáles podrían reutilizarse en otra plataforma.

6. Grado de avance

6.1. ¿Se finalizó el desarrollo completo?

El desarrollo del sistema de control se encuentra completo. Se implementaron y se integraron entre sí los cinco componentes principales: el sensor ultrasónico HC-SR04, el filtro de Kalman, el algoritmo PID, el servomotor MG90S y el potenciómetro lineal, todo ello sobre la arquitectura de tareas de FreeRTOS descrita en la sección anterior. El sistema hoy corre un lazo de control cerrado real: la posición del objeto deslizante se mide, se filtra, se compara contra el setpoint fijado por el usuario y se corrige actuando sobre el servomotor, todo de forma periódica y concurrente.

6.2. Funcionamiento correcto de los periféricos

Los tres periféricos de entrada/salida del sistema (sensor ultrasónico, servomotor y potenciómetro con su conversor ADC) fueron validados individualmente y luego verificados en conjunto, funcionando dentro del lazo de control completo.

El sensor ultrasónico HC-SR04 se integró mediante Input Capture por hardware: el driver dispara el pulso de trigger y mide el ancho del eco por interrupción, sin bloquear a la tarea que lo utiliza. Durante las pruebas se detectaron comportamientos propios del componente cerca del sensor (zona ciega y eco fantasma) que llevaron a ajustar la política de recorte de las lecturas, para no ocultarle al lazo de control el movimiento real del objeto (ver la sección de evolución del proyecto).

El servomotor MG90S se controla mediante una señal PWM de 50 Hz generada por un timer de hardware, con una interfaz de software que expone únicamente el ángulo deseado en grados. Se calibró su recorrido físico (10° a 170°, con 90° como posición horizontal de referencia) y se agregó una guarda de coherencia, verificada en tiempo de compilación, entre el rango de salida del PID y el recorrido permitido del servo. Durante el montaje se detectaron inconvenientes de ruido eléctrico y movimientos poco suaves asociados a la alimentación y a la calidad de las conexiones; se resolvieron por completo estañando los extremos de los cables, lo que eliminó las interferencias.

El potenciómetro lineal se lee mediante el conversor ADC del microcontrolador y su valor normalizado se mapea linealmente al mismo rango de posiciones que cubre el sensor sobre la barra, de modo que cualquier setpoint pedido por el usuario sea siempre alcanzable por el objeto deslizante.

6.3. Modelado sobre RTOS

Sí. El sistema completo se ejecuta sobre FreeRTOS, con la arquitectura de tareas, colas, semáforos e interrupciones descrita en la Figura 2 ya integrada y en funcionamiento, y no únicamente con tareas independientes sincronizadas por retardos, como ocurría en una etapa temprana del proyecto.

7. Evolución del proyecto

En esta sección contamos un poco cómo fue evolucionando el proyecto, y algunos de los cambios más importantes que fueron surgiendo en el camino. El diseño planteado en el anteproyecto se mantuvo estable en sus líneas generales, pero llegar a la implementación final llevó varias etapas y decisiones que no responden a errores de planificación, sino a cosas que solo se hicieron evidentes al trabajar sobre el hardware real.

Arrancamos probando el sensor y el servomotor por separado, cada uno con librerías básicas de ejemplo, para conocer su comportamiento real antes de integrarlos a nada. En paralelo, mientras nuestro compañero de Diseño Industrial empezaba a diseñar la maqueta, nosotros arrancamos con todo el diseño de tareas de FreeRTOS: timers, colas, queue sets, etc.

Cuando la maqueta quedó terminada, empezamos a montar todo en su lugar. El servomotor nos dio bastantes problemas en esta etapa, pero terminaron siendo, en su mayoría, errores de conexión y cableado, sumados a una mala configuración inicial de su librería.

Con el sistema ya armado, corregimos a mano los parámetros del filtro de Kalman y del PID hasta llegar a una respuesta que nos convenciera, buscando un amortiguamiento cercano al crítico. En esta etapa usamos bastante el puerto serie UART para debuguear los valores que íbamos obteniendo, lo que nos permitió encontrar varios bugs.

Cuando el lazo de control ya andaba más o menos bien, incorporamos el potenciómetro. Su librería original tenía una función para fijar un rango en centímetros y devolvía directamente la medida ya convertida, pero nos pareció un mal diseño: mezclaba la lectura del hardware con una decisión de la aplicación. La separamos en dos partes: un módulo que solo devuelve la lectura normalizada del potenciómetro, y una función aparte, reutilizable, que la convierte linealmente a la magnitud que necesitemos en cada caso.

También encontramos un bug importante que ocurría muy cerca del sensor: cuando el objeto quedaba pegado a él, el sensor devolvía constantemente mediciones erróneas y el sistema quedaba en una especie de loop silencioso, el programa seguía corriendo, pero TaskMotor dejaba de recibir ángulos nuevos y el servomotor no se movía más. Lo resolvimos distinguiendo explícitamente el motivo de la lectura inválida (zona ciega física del sensor, o un eco fantasma donde la señal de eco queda en alto hasta agotar el timeout del driver) de una lectura real por debajo del rango de confianza esperado, para no ocultarle al filtro de Kalman el movimiento real del objeto cerca del sensor. En la misma línea, en una etapa intermedia

eliminamos también un desplazamiento fijo que se usaba para convertir entre la distancia medida por el sensor (referida al borde del objeto) y la posición de su centro sobre la barra: hoy todo el lazo de control trabaja directamente sobre esa misma referencia de borde, sin ningún offset intermedio.

A medida que fuimos encontrando estos problemas, también reorganizamos el código: al principio todo vivía en una única carpeta, y terminamos separándolo en capas (drivers de hardware, algoritmos de control puros y las tareas que los conectan), simplificando de paso las interfaces de las librerías de cada periférico. Otro cambio importante fue pasarle a cada tarea, como argumento, el contexto que necesita para funcionar (sus colas, semáforos y handles), en vez de acceder a todo eso de forma global como hacíamos al principio.

Sobre el final, migramos todas las colas, semáforos y tareas a las variantes `Static` de FreeRTOS, menos el queue set del controlador PID, porque esta versión de FreeRTOS no tiene una variante estática para ese objeto. Más adelante nos dimos cuenta de que ni siquiera hacía falta esa cola: como TaskPID solo necesita recalcular cuando le llega una posición filtrada nueva, el setpoint del potenciómetro no necesitaba viajar por una cola ni despertar a nadie por sí solo. Terminamos reemplazando esa cola y el queue set por un `float` compartido, al que TaskPID accede por puntero, sacándonos de encima la última asignación dinámica de memoria del proyecto.

Por último, durante gran parte del desarrollo mantuvimos trazas de depuración por UART, pruebas automáticas embebidas y un modo de sensor sintético para poder validar el lazo de control sin depender del hardware físico. Una vez validado el funcionamiento del sistema, sacamos deliberadamente esa instrumentación de la rama de trabajo utilizada para este informe, dejando únicamente el código necesario para el funcionamiento en producción.

8. Métricas

Más allá de la validación funcional de cada componente, se realizaron pruebas de perturbación sobre el lazo de control ya integrado: con el sistema funcionando y el objeto estabilizado en el setpoint, se lo desplazaba manualmente a otra posición sobre la barra y se midió con cronómetro el tiempo que tardaba el sistema en detectar el error y volver a estabilizarlo. La prueba se repitió diez veces sobre tres zonas distintas de la barra: el centro, el extremo opuesto al sensor y la zona cercana al sensor.

En la gran mayoría de los ensayos la precisión de llegada al setpoint rondó el 5 % de error; se registraron algunos pocos valores atípicos, con errores cercanos al 10 %.

Ensayo	Tiempo de establecimiento (s)
1	2.85
2	2.75
3	2.57
4	2.55
5	2.43
6	2.68
7	2.87
8	2.45
9	2.83
10	2.77

Cuadro 2: Tiempo de establecimiento ante perturbación en el centro de la barra (media: 2.68 s).

Podemos notar que al sistema le cuesta mayor tiempo recuperarse cuando el setpoint se encuentra en alguno de los extremos de la barra. Una explicación posible para esto, es que estando en un extremo de la barra, las perturbaciones que podemos realizar son mucho mayores que si nos encontramos en el medio de la misma.

Ensayo	Tiempo de establecimiento (s)
1	3.34
2	3.15
3	3.46
4	3.24
5	3.11
6	3.35
7	2.81
8	2.84
9	3.27
10	3.13

Cuadro 3: Tiempo de establecimiento ante perturbación en el extremo opuesto al sensor (media: 3.17 s).

Ensayo	Tiempo de establecimiento (s)
1	3.80
2	3.45
3	3.50
4	2.94
5	3.85
6	3.25
7	3.01
8	2.58
9	2.45
10	2.93

Cuadro 4: Tiempo de establecimiento ante perturbación cerca del sensor (media: 3.18 s).

9. Construcción de la Maqueta

El diseño y la construcción de la maqueta estuvieron a cargo de Valentín Sabino, estudiante de la Licenciatura en Diseño Industrial de la Facultad de Arquitectura, Planeamiento y Diseño de la Universidad Nacional de Rosario. El diseño se modeló y se validó en Autodesk Inventor; en la Figura 3 se muestran algunas vistas de ese modelado.

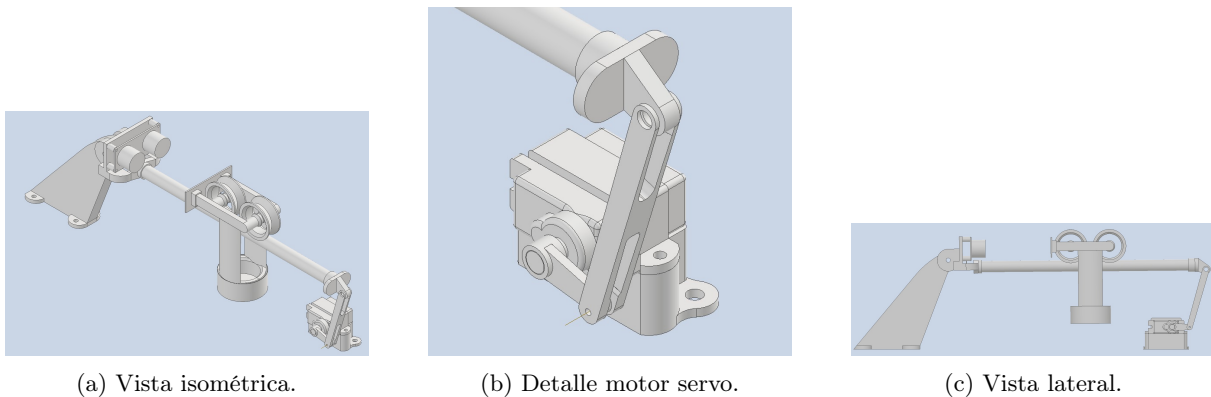


Figura 3: Imágenes del modelado de la maqueta

En una primera etapa del diseño, la maqueta contaba con dos rieles de madera y un carrito de cuatro ruedas que se desplazaba sobre ellos. Este diseño presentaba mucho rozamiento, lo que dificultaba que el carro comenzara a moverse. Por ese motivo se pasó al diseño actual: un único riel y un carro de dos ruedas, con un contrapeso ubicado por debajo del riel que facilita el arranque del movimiento.

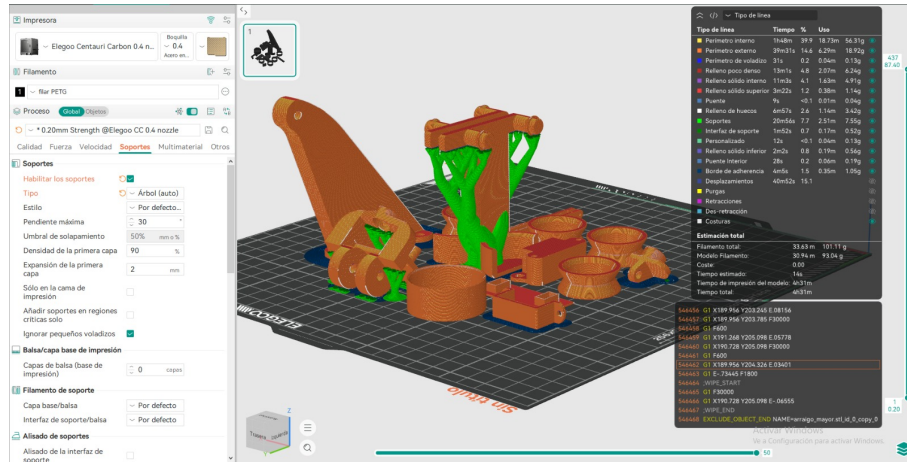


Figura 4: Preparación para la impresión de la maqueta.

Una vez finalizado el modelado, se procedió a la impresión de la maqueta. En la Figura 4 pueden verse las piezas a imprimir (en naranja) junto con los soportes generados automáticamente (en verde) para sostener la impresión durante el proceso.

La técnica de impresión 3D utilizada fue FDM (*Fused Deposition Modeling*), y el material elegido para las piezas estructurales fue PETG (*Polietileno Tereftalato Glicol*), por su buena resistencia mecánica. La tornillería utilizada fue estándar, y para la base se empleó una tabla de pino. El carro incorpora rodamientos (rulemanes) ABEC 8, y el contrapeso se resolvió con una pieza de acero inoxidable reciclada.

10. Conclusiones

El sistema logró cumplir el objetivo general planteado: un lazo de control cerrado en tiempo real que mide, filtra, corrige y actúa sobre la posición del objeto deslizante, con las cinco tareas de FreeRTOS descritas en la Sección 4 integradas y funcionando de forma concurrente sobre el microcontrolador.

En el camino aprendimos mucho sobre hardware y sobre cómo comunicarnos con él desde el software: la necesidad de distinguir explícitamente entre una lectura inválida del sensor (zona ciega, eco fantasma) y una lectura real fuera de rango, para no ocultarle al filtro de Kalman el movimiento real del objeto, fue solo un ejemplo de los detalles que solo se hacen evidentes al trabajar con hardware real. También fue la primera vez que programamos algo con un accionar directo en el mundo físico, en lugar de un programa que se mantiene dentro de la computadora sin efectos por fuera de ella.

Como se plantea en el objetivo general (Sección 2.1), se buscaba posicionar y mantener el objeto de forma precisa, eficiente y en el menor tiempo posible. Si bien el sistema cumple con esto, las métricas de la Sección 8 muestran margen de mejora en los tres frentes: reducir la frecuencia de los casos atípicos que llegaron a un error cercano al 10 %, y acortar el tiempo de establecimiento observado (entre 2.7 y 3.2 segundos según la zona de la barra), posiblemente ajustando con más cuidado las ganancias del PID y el filtro de Kalman. Aun así, consideramos que se llegó a un resultado muy satisfactorio.